

Lab 16: File Operations & I/O Exception Handling

Recap: Lab (Github) Workflow - How to Work on Labs

Follow these steps for every lab carefully to access, complete, and submit your assignment.

1. Accept the Assignment

- Open the Lab Assignment Link the professor provided.
- Click "**Accept the assignment**". This will create your personal assignment repository on GitHub under the **00P-Fall-2025** organization.
- You'll be taken to your repository page. Verify that the URL looks like **github.com/00P-Fall-2025/lab-number-yourusername**.

2. Clone the Repository to Your Computer

- On your repository page, click the blue **<> Code** button.
- In the dropdown menu, choose "**Open with GitHub Desktop**".
- GitHub Desktop will launch. Choose a preferred local folder on your computer to save the project and click "**Clone**".
- If asked "How are you planning on using this fork?", select "**For my own purpose**" and continue.

3. Open in VS Code and Start Coding

- In GitHub Desktop, ensure the "Current repository" is the one for this lab.
- Click the "**Open in Visual Studio Code**" button.
- VS Code will open the project folder. You can now begin writing your solutions in the **Lab16.java** file.

4. Save and Submit Your Work

- **Commit (Save) Changes:** As you work, save your file in VS Code (**Ctrl+S** or **Cmd+S**). To record your progress, go to the **Source Control** tab (the fork icon) on the left sidebar in VS Code. Type a descriptive message in the message box (e.g., "Completed Lab 16") and click "**Commit**". You must enter a message.
- **Push (Submit) to GitHub:** When you are finished with the lab or want to back up your work, go back to GitHub Desktop. Click the "**Push origin**" button at the top of the window. This sends your committed changes from your computer to your GitHub repository online.

5. Verify Your Submission

- After you push, you can click "**View on GitHub**" in GitHub Desktop to open your repository in the browser.
- On the GitHub website, make sure you are viewing the **main** branch and confirm that all of your latest code is visible.

Lab 16: File Operations & I/O Exception Handling

Lecture 16 Concept Recap

Concept	Class/Method	Purpose	Example
File Creation	<code>File.createNewFile()</code>	Create a new file on disk	<code>myFile.createNewFile()</code> → returns true if created
File Existence	<code>File.exists()</code>	Check if file already exists	<code>if (file.exists())</code> → prevents duplicates
File Deletion	<code>File.delete()</code>	Remove a file	<code>file.delete()</code> → returns true if deleted
Directory Creation	<code>File.mkdir()</code>	Create a folder	<code>myFolder.mkdir()</code>
List Files	<code>File.listFiles()</code>	Get all files in a folder	<code>file.listFiles()</code> → returns array of Files
Write Data	<code>FileWriter</code>	Write text to file (overwrites)	<code>new FileWriter(file)</code>
Append Data	<code>FileWriter</code> (append mode)	Add text without overwriting	<code>new FileWriter(file, true)</code>
Efficient Write	<code>BufferedWriter</code>	Write larger chunks faster	Wrap <code>FileWriter</code> inside <code>BufferedWriter</code>
Read Data	<code>FileReader</code>	Read file character by character	<code>fileReader.read()</code> → returns int as char
Efficient Read	<code>BufferedReader</code>	Read file line by line	<code>bufferedReader.readLine()</code> → returns String
Handle Errors	<code>try-catch-finally</code>	Handle <code>IOException</code> safely	Always close files in finally block
Propagate Errors	<code>throws</code>	Let caller handle exceptions	<code>method() throws IOException</code>

Part 1: Game Save System

Real-World Scenario

Build a game where players can save progress. System must create save directories, write player data efficiently, and append game logs.

Key Concepts to Practice

- ✓ Directory creation with `.mkdir()`
- ✓ File existence check with `.exists()`
- ✓ File creation with `.createNewFile()`
- ✓ FileWriter for writing data
- ✓ BufferedWriter for efficient writing
- ✓ FileWriter append mode (parameter = true)
- ✓ Try-catch-finally for IOException

What to Complete

1. `createSaveDirectory()` method - Add try-catch-finally:

- **Try block:**
 - Create File object representing "saves" directory
 - Check if directory exists using `.exists()` method
 - If it doesn't exist, create it using `.mkdir()`
 - Set `operationSuccessful = true`
- **Catch IOException:**
 - Print error message with exception details
- **Finally block:**
 - Always display operation status

2. `savePlayerData()` method - Add try-catch-finally:

- **Try block:**
 - Create File object for the file path
 - Use `.createNewFile()` to create file if not exists
 - Create FileWriter object
 - Create BufferedWriter (wrap the FileWriter inside it)
 - Use `bufferedWriter.write()` to write header and player data
 - Use `bufferedWriter.newLine()` to add line breaks
 - Close both BufferedWriter and FileWriter
 - Print success message
 - Set `operationSuccessful = true`
- **Catch FileNotFoundException:**
 - Invalid file path or directory doesn't exist
 - Print: "X Error: Invalid file path. Cannot save game."
- **Catch IOException:**
 - Write operation failed (disk full, permissions, etc.)
 - Print error with exception message
- **Finally block:**

- Always display save status

3. `appendGameLog()` method - Add try-catch-finally:

- **Try block:**

- Create FileWriter with **append mode**: `new FileWriter(filename, true)`
- The second parameter `true` means append (don't overwrite)
- Write log entry: `"\n[LOG] " + logEntry`
- Close the FileWriter
- Print: `"✓ Log entry added: [logEntry]"`
- Set `operationSuccessful = true`

- **Catch IOException:**

- Print error with exception message

- **Finally block:**

- Always display append status

Testing Part 1

- ☐ "saves" directory created → ✓ Directory SUCCESS
- ☐ Player data saved to file → ✓ Save SUCCESS
- ☐ Log entry appended to file → ✓ Append SUCCESS
- ☐ File exists before saving → Works without re-creating
- ☐ Game data persisted in file → Can verify in saves/game_save.txt

Part 2: Employee Management System

Real-World Scenario

HR department manages employee database files. System must read employee records, check file existence, list directory contents, and clean up old backups.

Key Concepts to Practice

- ✓ File existence check with `.exists()`
- ✓ Directory existence and creation
- ✓ FileReader for reading character-by-character
- ✓ BufferedReader for reading line-by-line
- ✓ `.readLine()` to get full lines as strings
- ✓ `.listFiles()` to list all files in directory
- ✓ File deletion with `.delete()`
- ✓ Throws keyword to propagate exceptions

What to Complete

In `manageEmployeeFiles()` method - Add throws and implementation:

1. Add throws signature:

- Add `throws FileNotFoundException, IOException` to method signature

2. Check/Create directory:

- Create File object for "employees" directory
- Use `.exists()` to check if it exists
- If it doesn't exist, use `.mkdir()` to create it
- Print creation message if needed

3. Check if file exists:

- Create File object for the employee file (parameter: filename)
- Use `.exists()` to verify file exists
- If file doesn't exist, throw: `new FileNotFoundException("Employee file not found: " + filename)`
- Don't use try-catch here - let the exception propagate to main()

4. Read file with BufferedReader:

- Create FileReader object (may throw FileNotFoundException)
- Create BufferedReader (wrap FileReader inside it)
- Use `.readLine()` to read first line as a string
- Print: "✓ First employee record: " + firstLine
- Close both readers

5. List files in directory:

- Use `.listFiles()` on the employees directory
- Loop through array and print each filename
- Handle null case (if directory doesn't exist)

6. Delete old backup:

- Create File object for "employees/employee_backup.txt"
- Use `.exists()` to check if backup exists
- If it exists, use `.delete()` to remove it
- Print confirmation if deleted

In `main()` method (part2) - Add try-catch:

- Call `EmployeeManager.manageEmployeeFiles("employees/employee_data.txt")`
- **Catch FileNotFoundException:**
 - Print: "× Error: Employee file not found in database"
- **Catch IOException:**
 - Print: "× Error: Unable to read employee file. " + e.getMessage()

- ☐ Directory created if doesn't exist → ✓ Created employees directory
- ☐ File existence checked → ✓ Employee file found
- ☐ First line read using BufferedReader → ✓ First employee record: [data]
- ☐ All files listed → ✓ Files in employees directory
- ☐ Old backup deleted if exists → ✓ Old backup deleted
- ☐ FileNotFoundException caught → ✗ File not found error
- ☐ IOException caught → ✗ Read error

Key Differences: Writing vs Reading

Operation	Overwrites?	Method	Example
Write (new)	Yes	FileWriter	new FileWriter("data.txt")
Append	No	FileWriter with true	new FileWriter("data.txt", true)
Efficient Write	Depends	BufferedWriter	new BufferedWriter(fileWriter)
Read chars	N/A	FileReader.read()	int ch = reader.read() → cast to char
Read lines	N/A	BufferedReader.readLine()	String line = bufferedReader.readLine()

Important Methods Reference

```
// File Creation & Management
File myFile = new File("filename.txt");
myFile.createNewFile();           // Creates file, returns true/false
myFile.exists();                 // Checks if exists, returns true/false
myFile.delete();                 // Deletes file, returns true/false

// Directory Management
File myDir = new File("folder_name");
myDir.mkdir();                   // Creates directory, returns true/false
File[] files = myDir.listFiles(); // Gets array of all files in directory

// Writing (OVERWRITES by default)
FileWriter writer = new FileWriter("file.txt");
writer.write("Hello");
writer.close();

// Appending (DOESN'T overwrite)
FileWriter writer = new FileWriter("file.txt", true); // true = append mode
writer.write("More data");
writer.close();
```

```
// Writing Efficiently (Buffered)
BufferedWriter buffered = new BufferedWriter(new FileWriter("file.txt"));
buffered.write("Line 1");
buffered.newLine();
buffered.close();

// Reading Characters
FileReader reader = new FileReader("file.txt");
int character = reader.read();    // Returns -1 at end of file
char ch = (char) character;      // Cast int to char
reader.close();

// Reading Lines (Buffered - More efficient)
BufferedReader buffered = new BufferedReader(new FileReader("file.txt"));
String line = buffered.readLine(); // Returns null at end of file
buffered.close();
```

Exception Handling Summary

Checked Exceptions (must catch):

- `FileNotFoundException` - File doesn't exist
- `IOException` - General I/O failure

Two Approaches:

1. **Try-Catch:** Method handles its own errors
2. **Throws:** Method lets caller handle errors

Always Remember:

- Close files in finally block OR use try-with-resources
- Always check `.exists()` before reading
- Use append mode `true` to prevent overwriting
- `BufferedReader/BufferedWriter` for efficiency